

EQP - Design Patterns

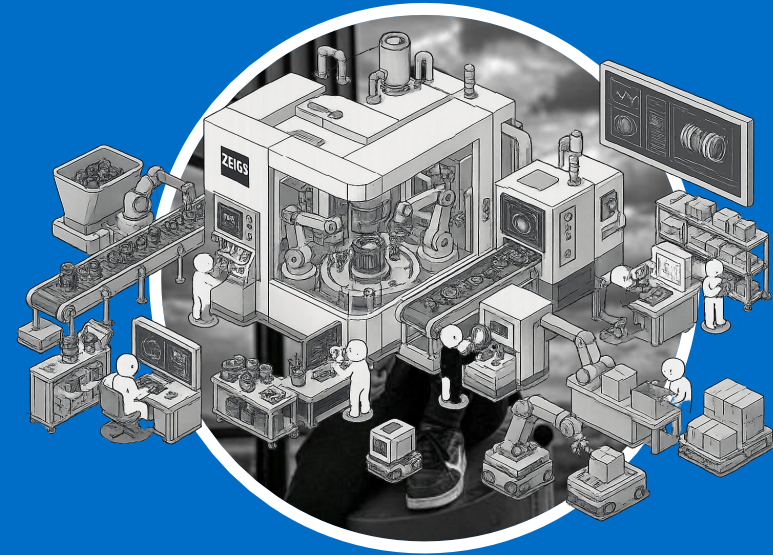
Same Problem – Different Chaos



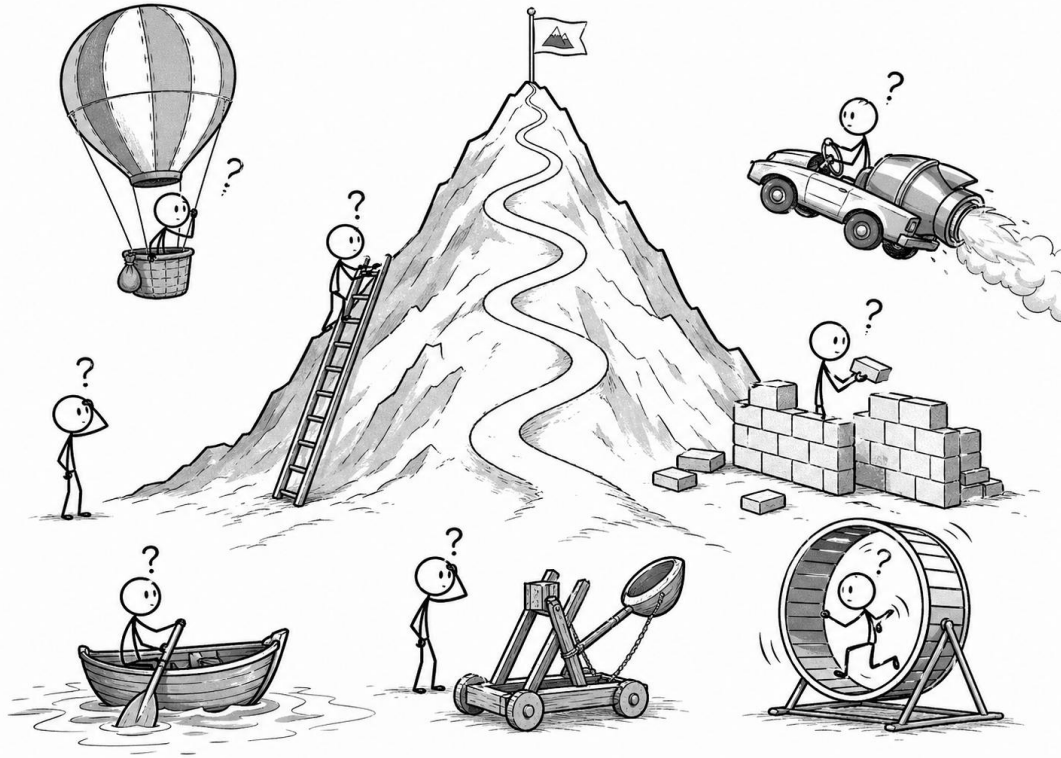


Jan-Hendrik Heinbokel

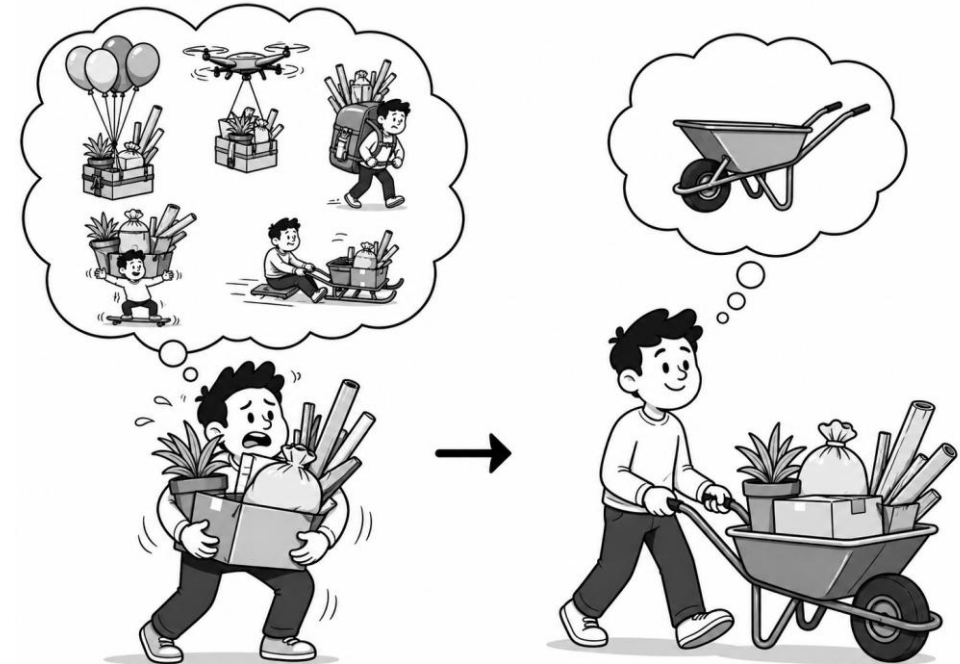
Senior Software Engineer · adesso SE
Software Architect im Projekt bei Carl Zeiss SMT



Was sind Design Patterns – und wozu braucht man sie?

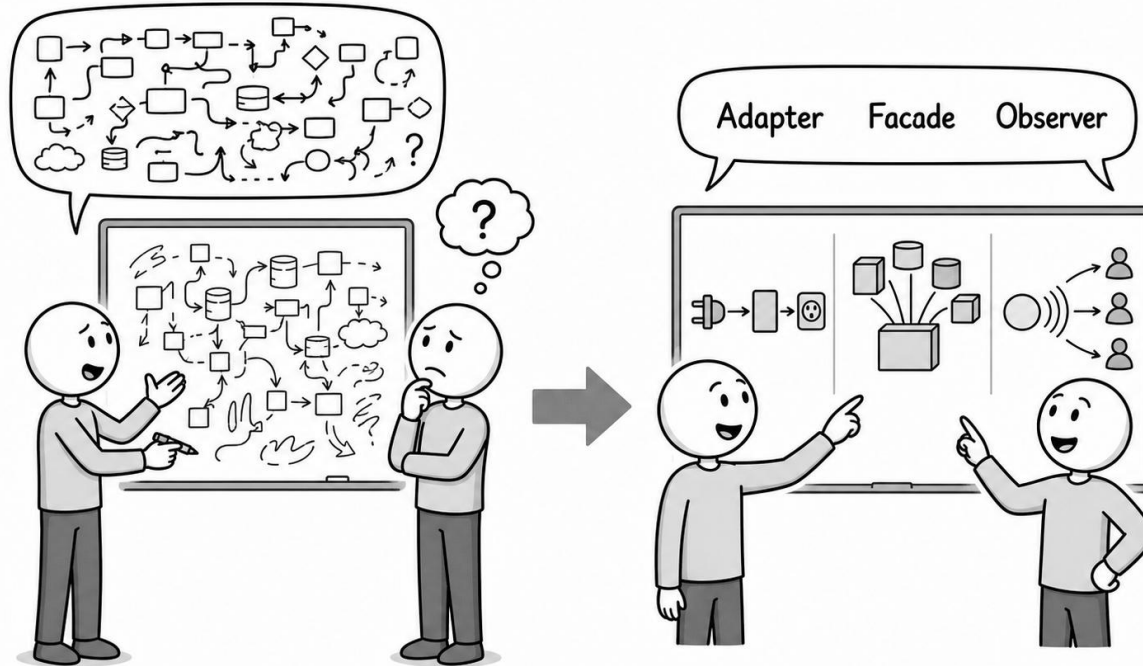


Einheitlichkeit



Wiederverwendbarkeit

Was sind Design Patterns – und wozu braucht man sie?



Gemeinsame Sprache

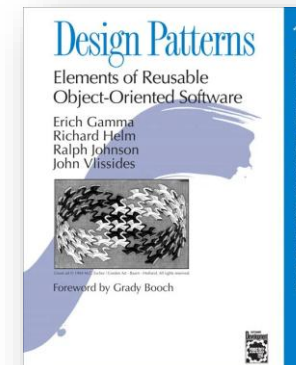
Was sind Design Patterns – und wozu braucht man sie?

Definition

- Bewährte, **wiederverwendbare Lösungsschablonen** für wiederkehrende Software-Designprobleme
- Keine fertigen Code-Bibliotheken, sondern beschriebene **Konzepte** und **Strukturen**
- Erstmals systematisiert von der „Gang of Four“ (Gamma, Helm, Johnson, Vlissides) – 1994

Warum Design Patterns?

- **Gemeinsame Sprache** im Team: Patterns sind benannte Konzepte, die Kommunikation vereinfachen
- **Wartbarer Code**: Fördern lose Kopplung, hohe Kohäsion und klare Verantwortlichkeiten
- **Rad nicht neu erfinden**: Jahrzehnte kollektiver Erfahrung direkt anwendbar



Erwartungshaltung

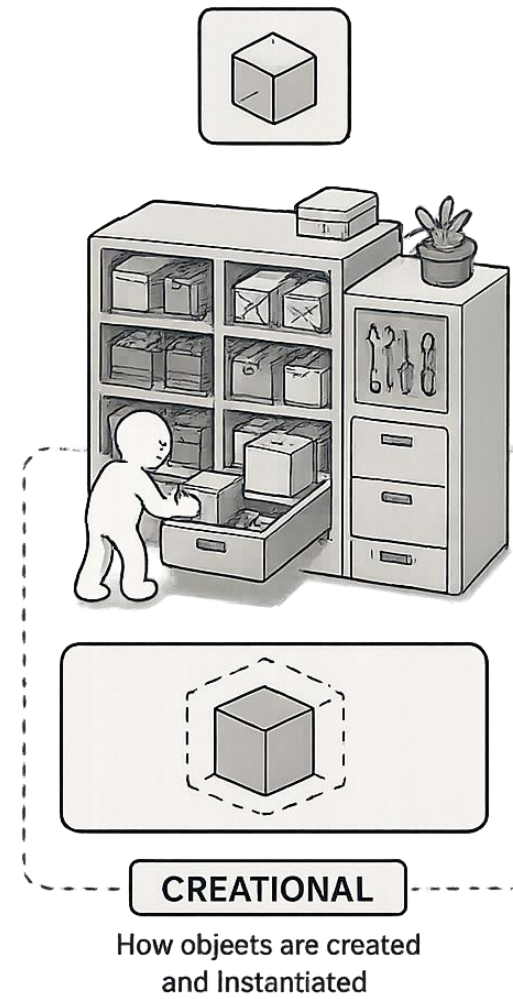
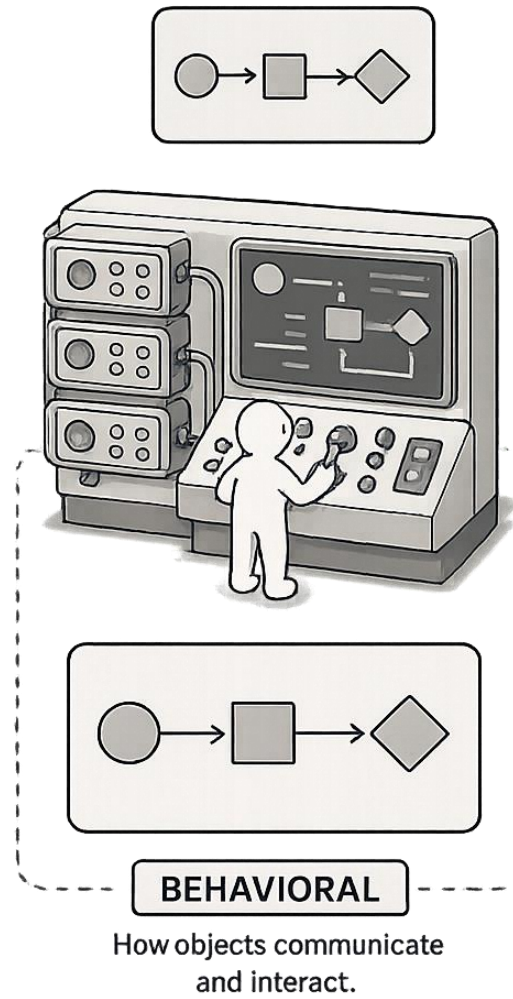
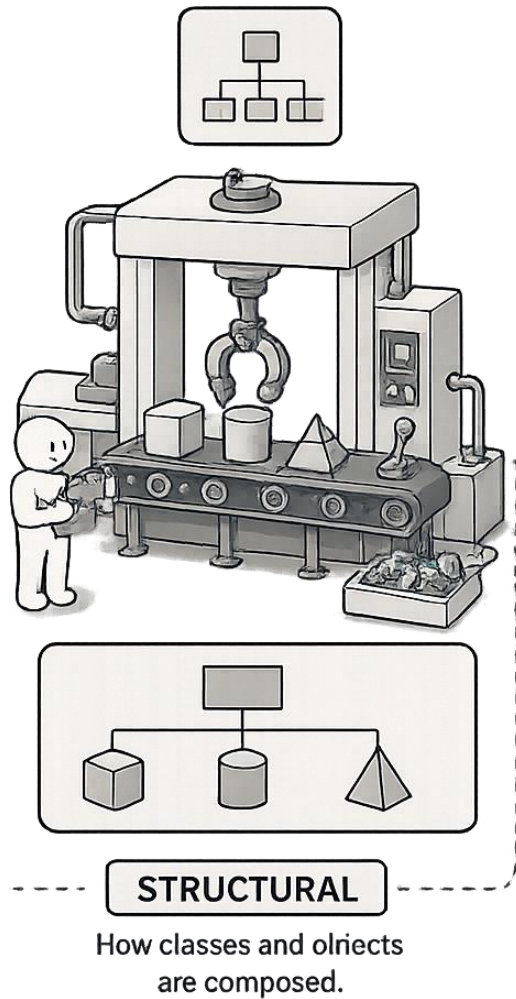
Was Patterns euch bringen

- Eigenen Code **einfach strukturierter** schreiben
- Patterns in Code schneller **erkennen und zuordnen**
- Schlagwörter in Architekturdiskussionen o. Ä. **verstehen und mitdiskutieren**

Was Patterns euch NICHT bringen

- **Kein Selbstzweck:** Patterns alleine bringen einen nicht weiter
- **Keine Pflicht:** Viele Probleme benötigen keine Patterns
- **Kein Qualitätssiegel:** Patterns machen Code nicht automatisch besser – gerne auch mal schlechter!
- Einige Patterns werdet ihr selber schreiben, viele nur erkennen und nutzen – **beides ist wichtig!**

Design Patterns – Welche Arten gibt es?



Design Patterns – welche gibt es?

Erzeugungsmuster

Creational Patterns

Steuern die Erzeugung von Objekten. Sie entkoppeln den Client von konkreten Klassen und machen die Instanziierung flexibel und wiederverwendbar.

- **Singleton** — Genau eine Instanz
- **Factory** — Erzeugung auslagern
- ...

Strukturmuster

Structural Patterns

Beschreiben, wie Klassen und Objekte zu größeren Strukturen zusammengesetzt werden, ohne deren Schnittstellen zu verändern.

- **Adapter** — Schnittstellen kompatibel machen
- **Decorator** — Verhalten beliebig schichten
- **Facade** — Subsysteme verstecken
- ...

Verhaltensmuster

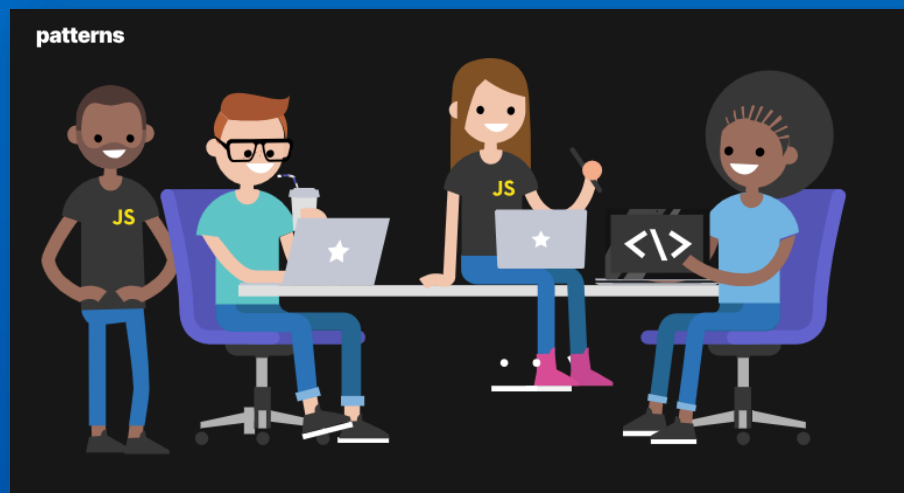
Behavioral Patterns

Regeln die Kommunikation und Verantwortung zwischen Objekten und machen Algorithmen austauschbar und erweiterbar.

- **Observer** — Benachrichtigungen ohne Empfänger zu kennen
- **Strategy** — Verhalten austauschbar machen
- ...



REFACTORING
• GURU •



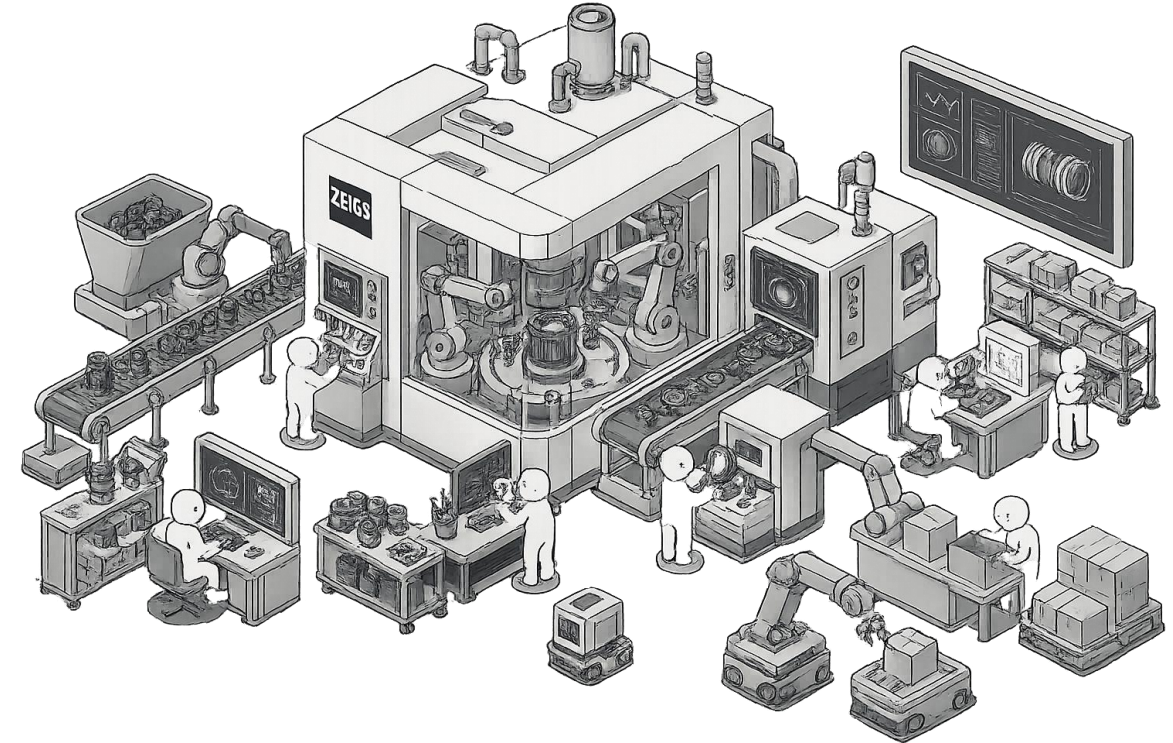
— LensForge — die Maschine als Lernfeld

Was ist das hier?

LensForge ist eine simulierte Fertigungsanlage für optische Linsen. Sie läuft, produziert „Teile“ — und hat Architekturprobleme, die wir gemeinsam lösen.

Wie arbeiten wir damit?

- Jede Kata erweitert die Maschine um ein echtes Feature
- Der fachliche Kontext bleibt immer gleich
- Die technischen Kontexte sind getrennt
 - Wer aussteigt wird nicht abgehängt



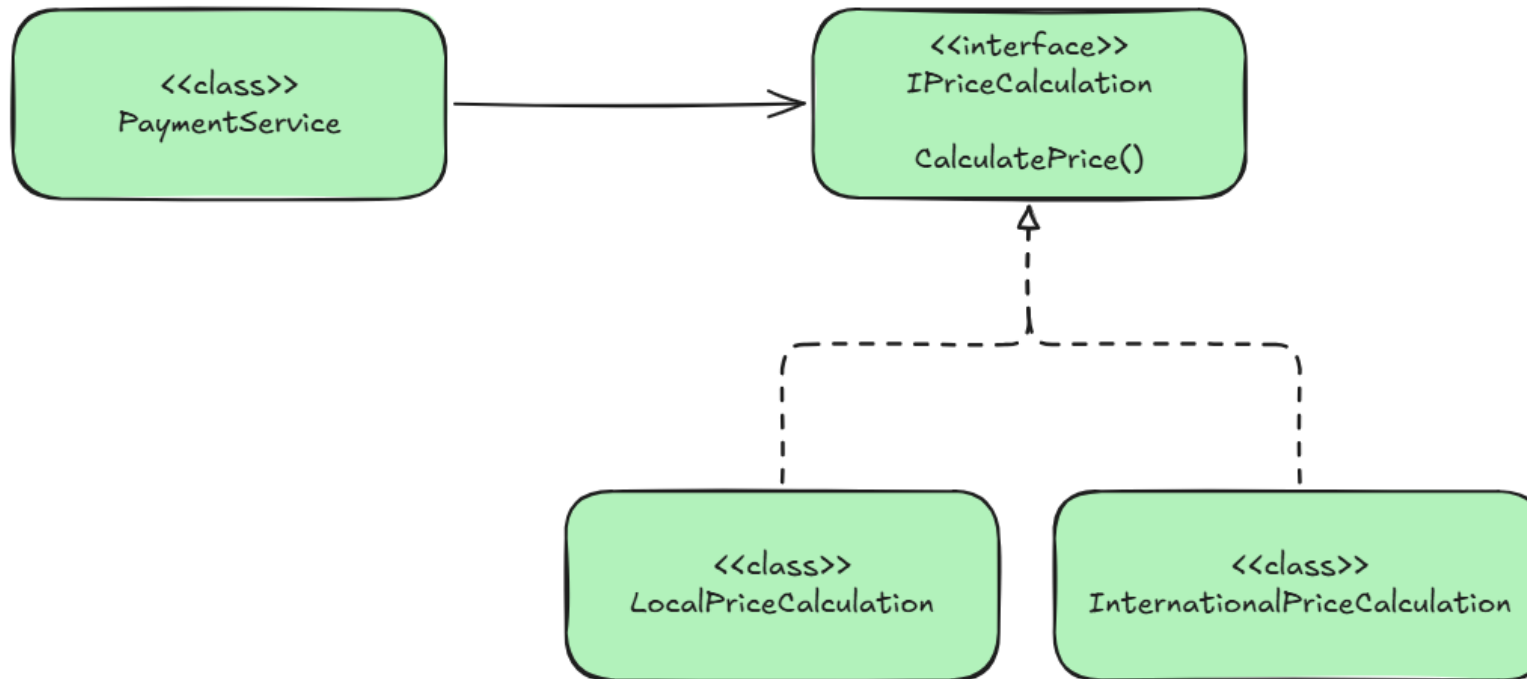
Pause



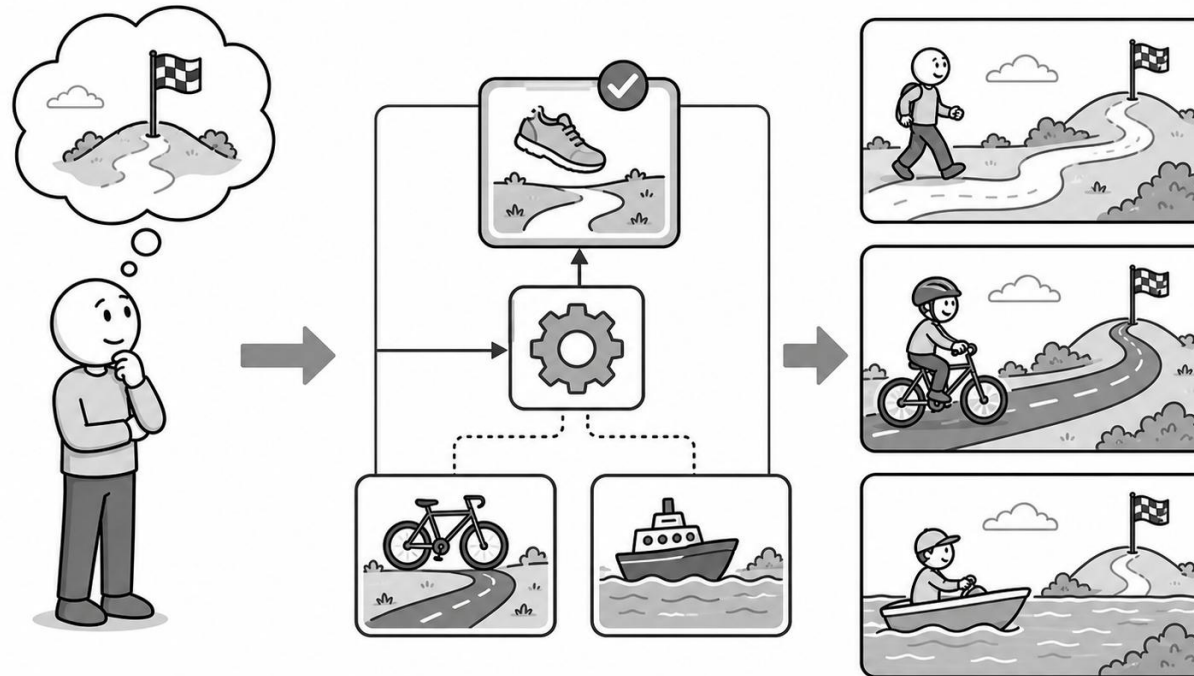
○ Praxisübung – Kata 01: Coating

- > **Coating | LensForge.Kata01/README.md**
- > Refactored CoatingStation.cs und implementiert Blue-Light-Filter (Standard + Premium) sowie Mirror-Coating!
- > ApplyCoating darf danach max. 10 Zeilen haben
- > Neue Verfahren dürfen bestehenden Code nicht anfassen
- > Jedes Verfahren muss isoliert testbar sein

○ Strategy



Strategy

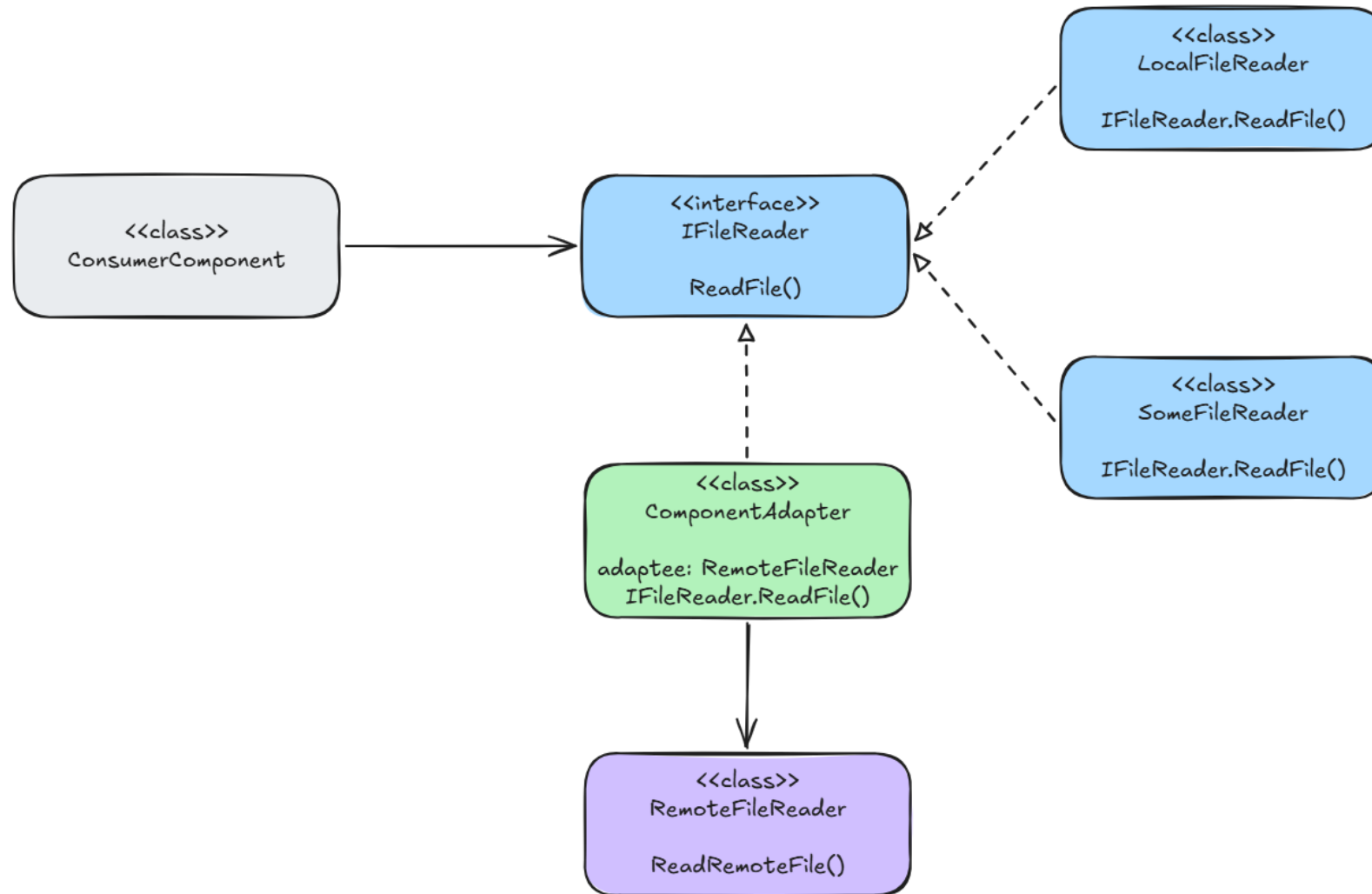


“Verhalten austauschbar machen, indem man es hinter einem gemeinsamen Interface kapselt.”

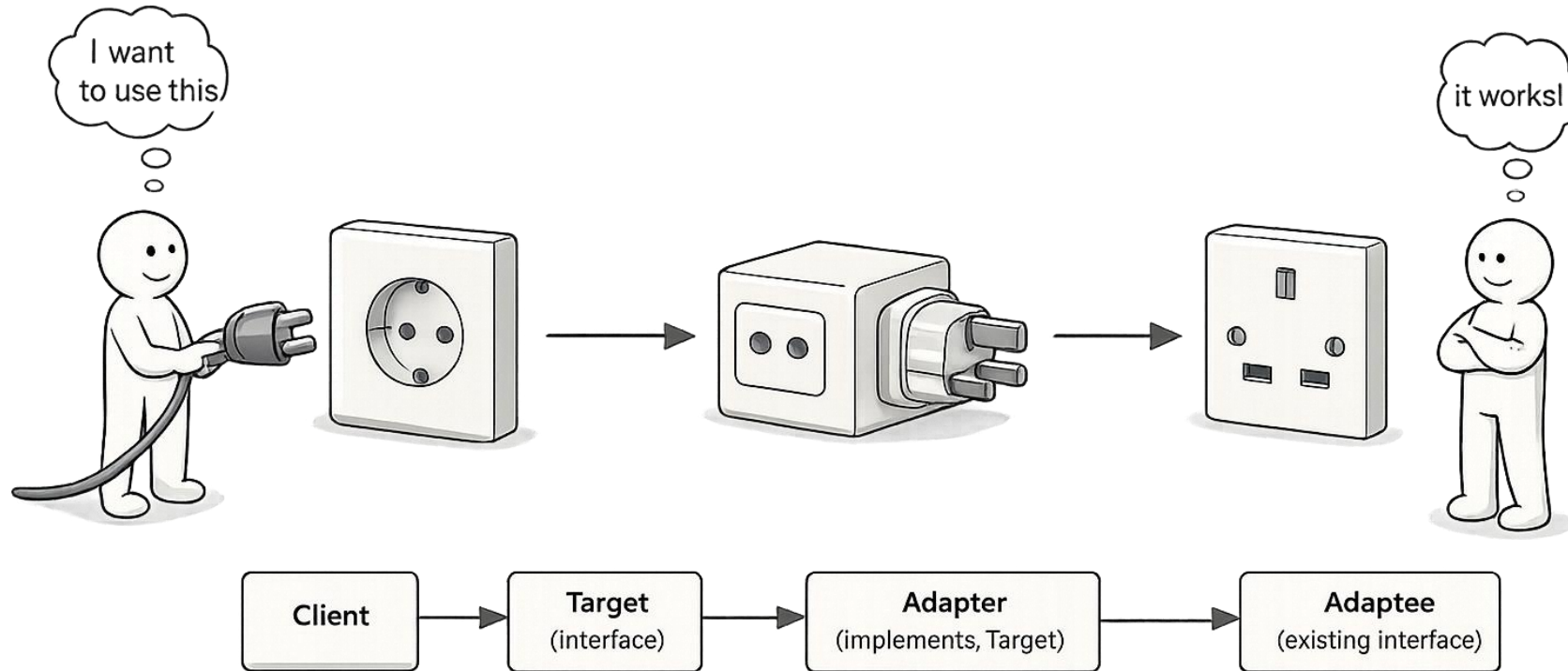
○ Praxisübung – Kata 02: Legacy-Schleifmaschine

- > **Legacy-Schleifmaschine | LensForge.Kata02/README.md**
- > Integriert die Legacy-Schleifmaschine als normale Station in die ProductionLine!
- > Program.cs soll sie durchlaufen!
- > LegacyGrindingSystem.cs, ProductionLine.cs und IProductionStep werden nicht angefasst!
- > Mit mindestens 2 Tests absichern!

Adapter



Adapter



“Eine inkompatible Komponente mit einem Interface kompatibel machen”

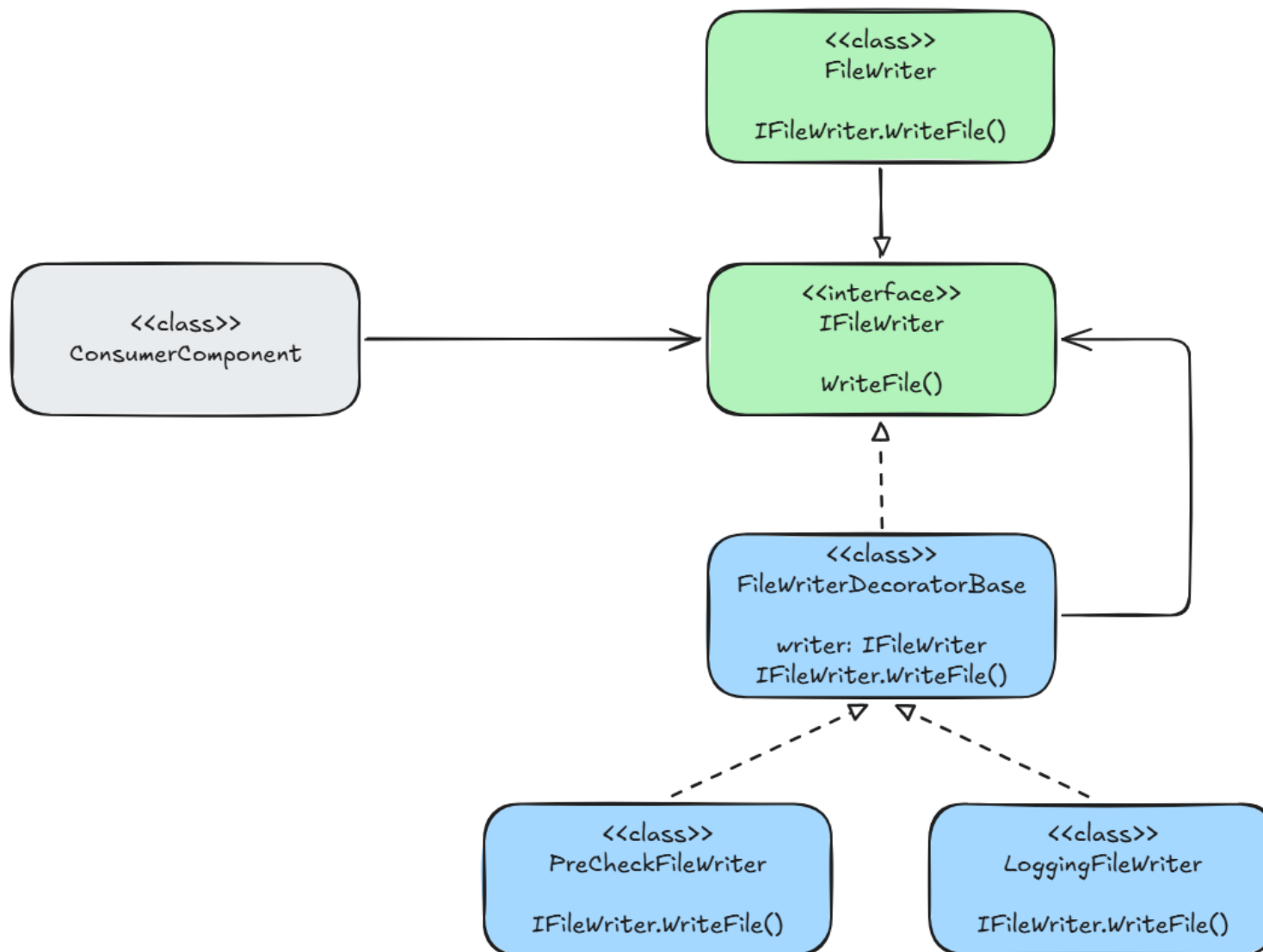
Pause



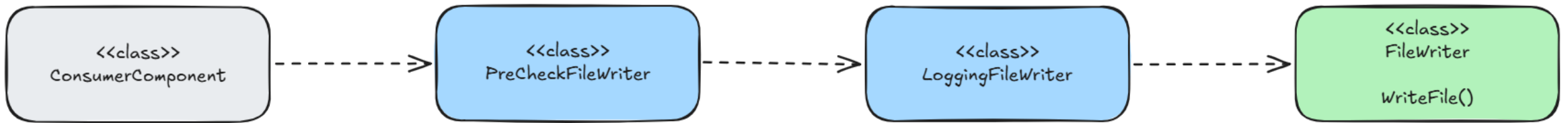
Praxisübung – Kata 03: QA-Pipeline

- > **QA-Pipeline | LensForge.Kata03/README.md**
- > Implementiert die drei Zusatzprüfungen so, dass alle drei Kunden-Szenarien in Program.cs korrekt durchlaufen!
- > BasicQaInspector und IQaInspector nicht anfassen!
- > Kein if kunde == "A" !
- > Jede Prüfung muss allein einsetzbar sein!

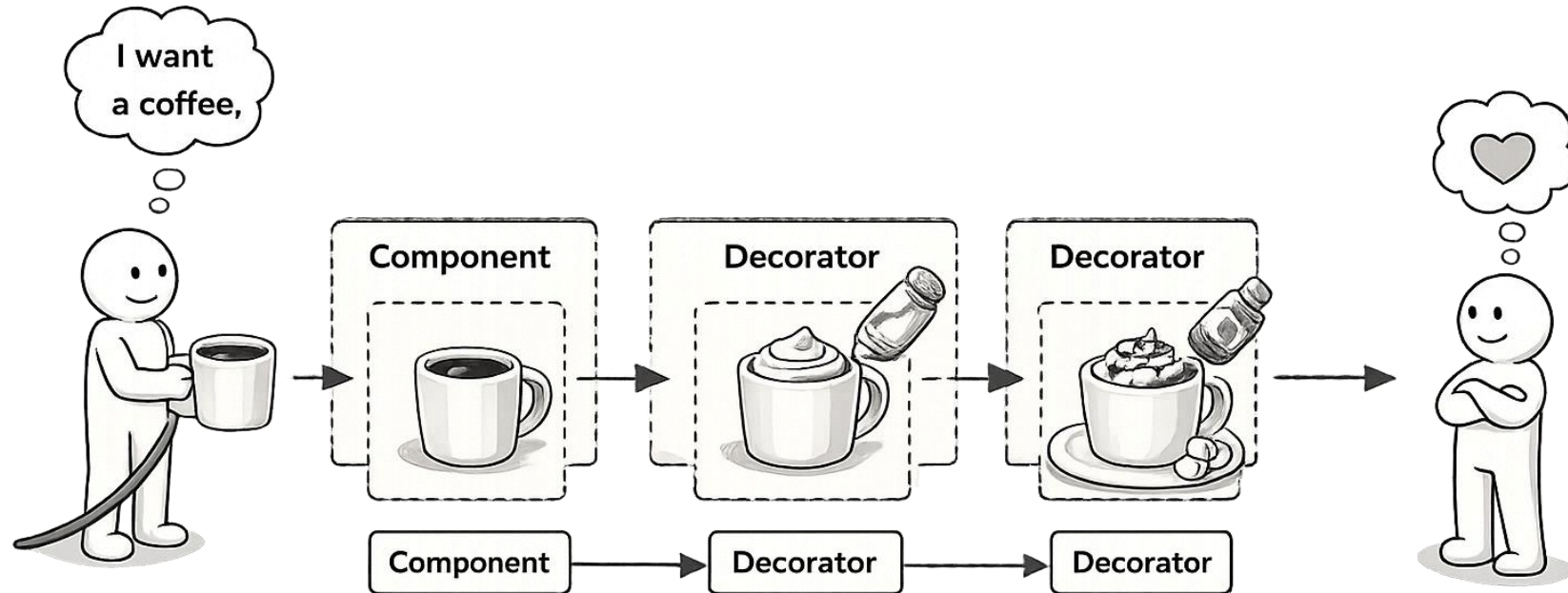
Decorator



Decorator



Decorator



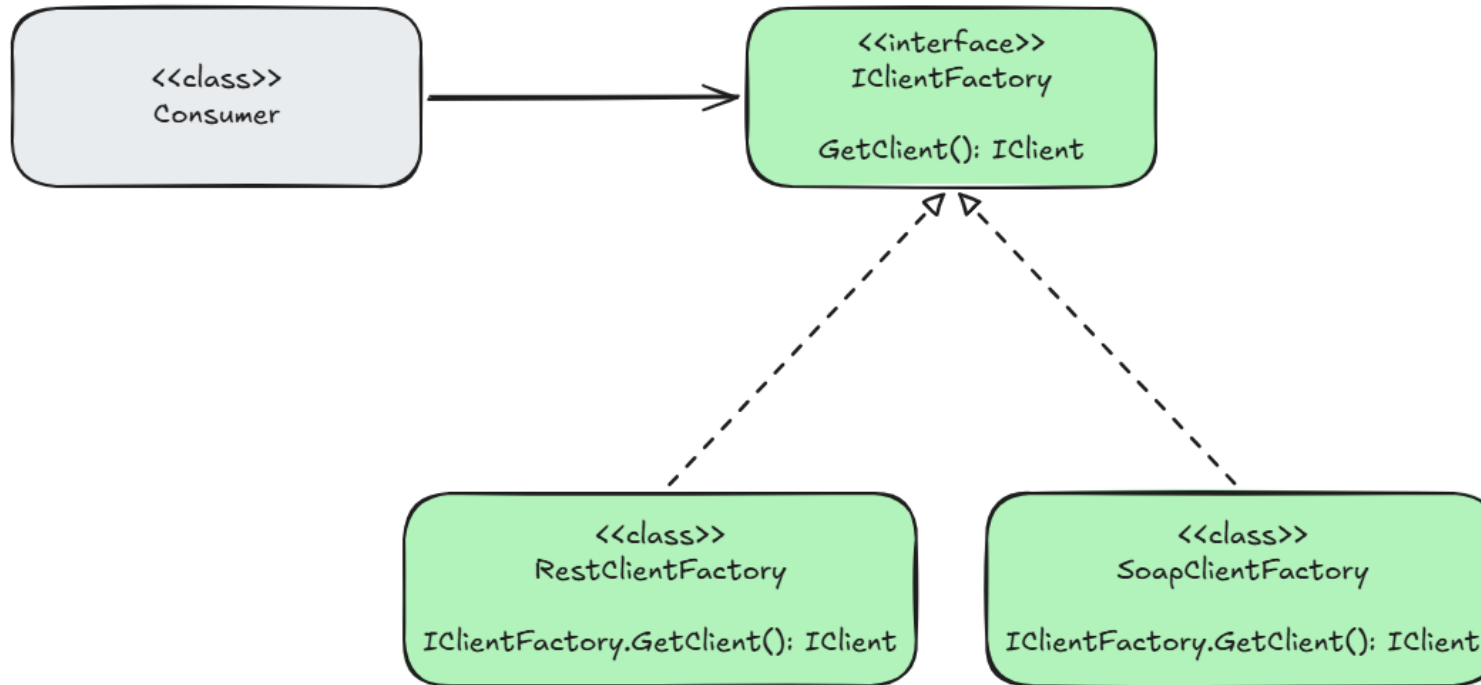
“Verhalten dynamisch um ein Objekt herum schichten, ohne die ursprüngliche Klasse zu ändern.”

○ Praxisübung – Kata 04: Verpackungs-Modul

- > **Verpackungs-Modul | LensForge.Kata04/README.md**
- > Beseitigt die duplizierte Logik in ShippingPreparation, OrderProcessor und InvoiceWriter!
- > Implementiert ExpressSecureBox und ReinforcedExportCrate!

- > Bestehende Verpackungsklassen und IPackaging nicht anfassen!
- > new PappkartonPackaging() & Co. nur noch an einer einzigen Stelle!

○ (Abstract) Factory



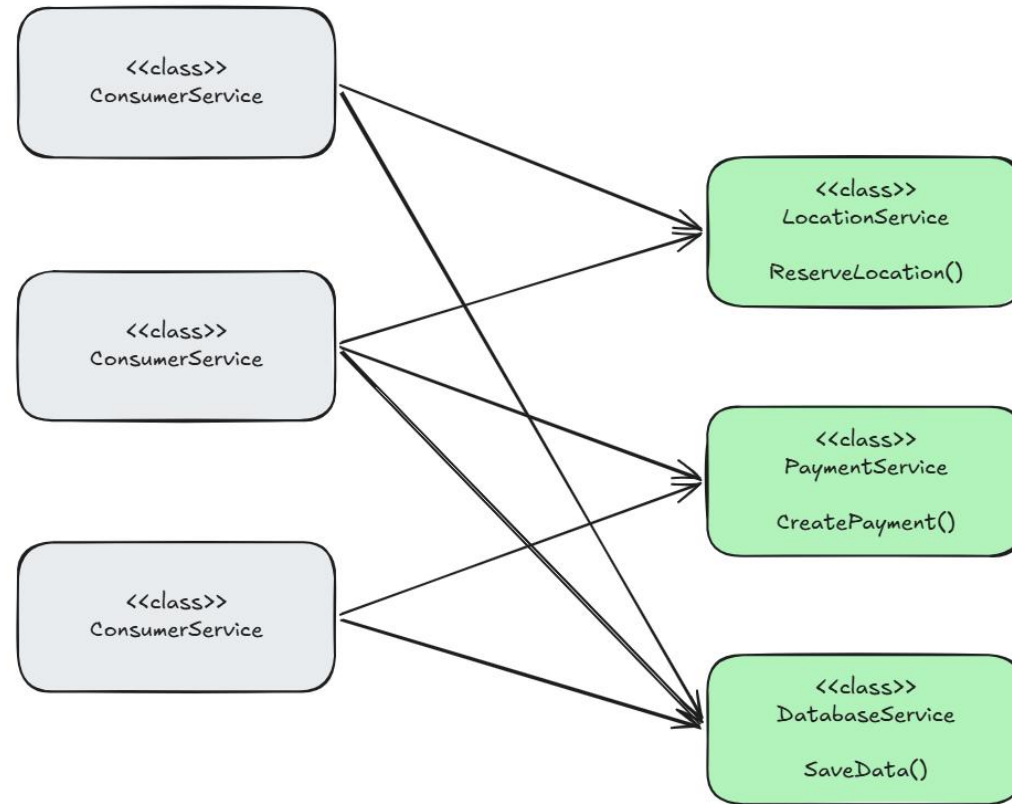
Pause



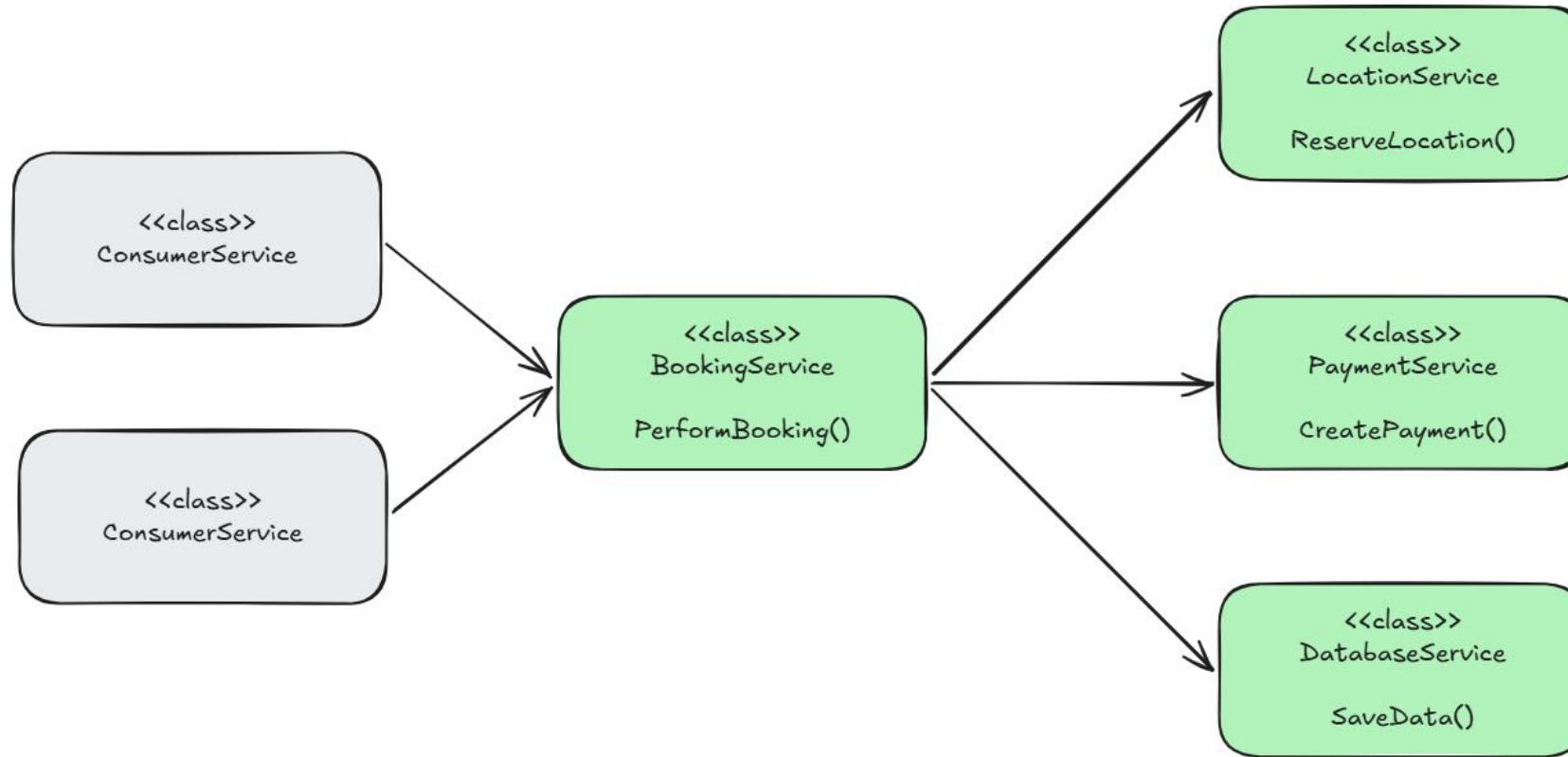
Praxisdemo

> Facade

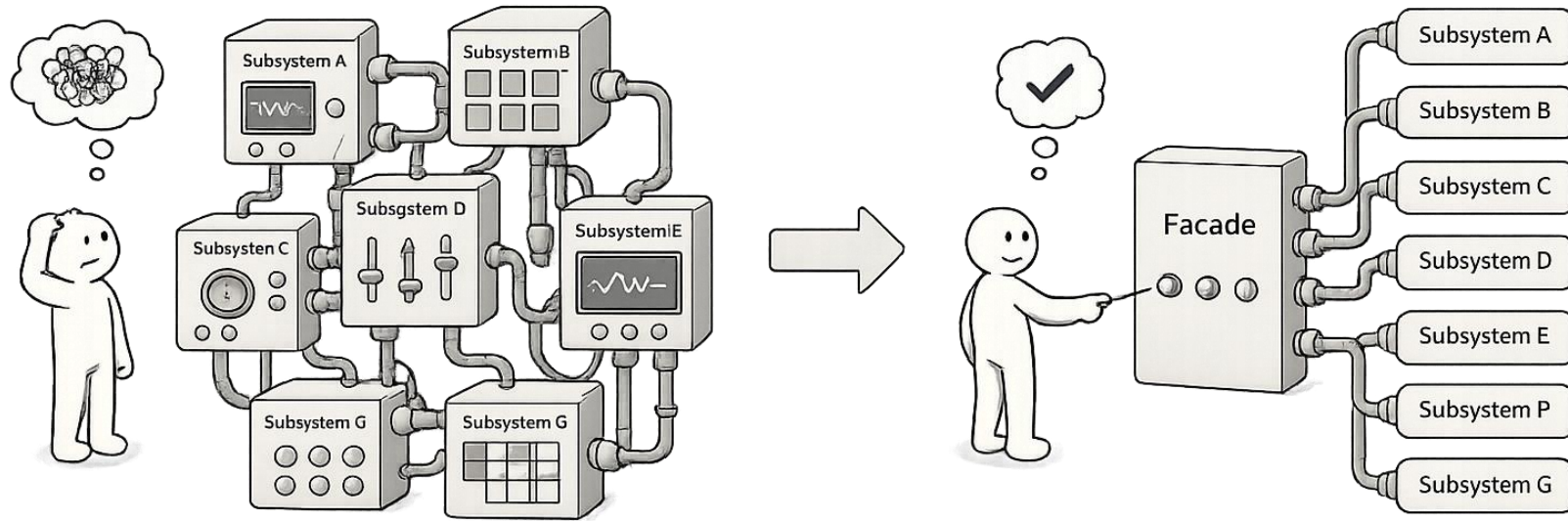
Ist-Situation



Facade



Facade



“Eine vereinfachte Schnittstelle vor ein komplexes System legen.”

Feierabend!

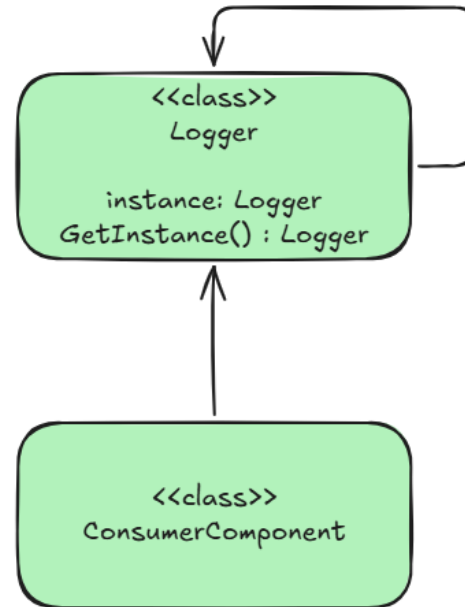


○ Praxisübung – Kata 05: Logger

- > **Logger-Singleton | LensForge.Kata05/README.md**
- > Refactored ProductionLogger auf genau eine Instanz!
- > Passt AssemblyLine, MaintenanceUnit und QualityGate entsprechend an!

- > Kein new ProductionLogger() bei Aufrufen!
- > Log-Level-Änderungen müssen sofort überall greifen!

Singleton



„Sicherstellen, dass es von einer Klasse genau eine geteilte Instanz gibt.“

○ Warum Singletons gefährlich sind

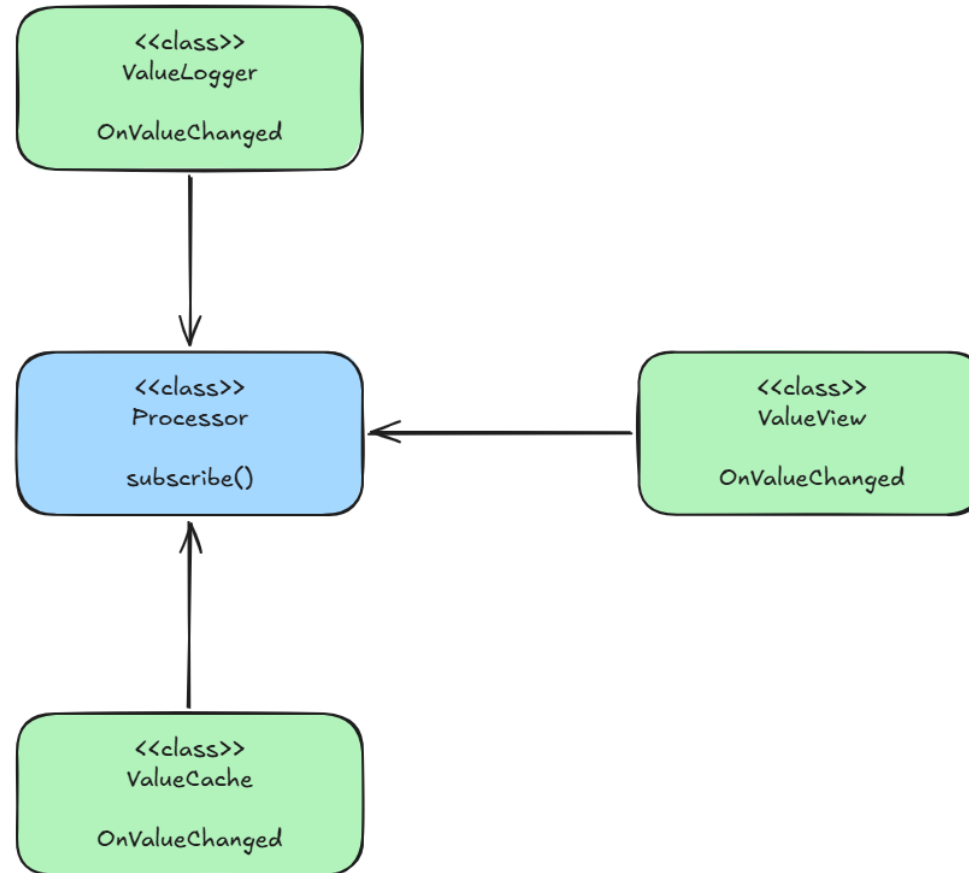
Was haben wir hier geschaffen?

- Direkte Kopplung an eine konkrete andere Komponente
- Kopplung an einen globalen Zustand
- Nicht mehr sauber einzeln testbar
- Abhängigkeiten versteckt
- Spätere Scope-Einschränkungen erschwert

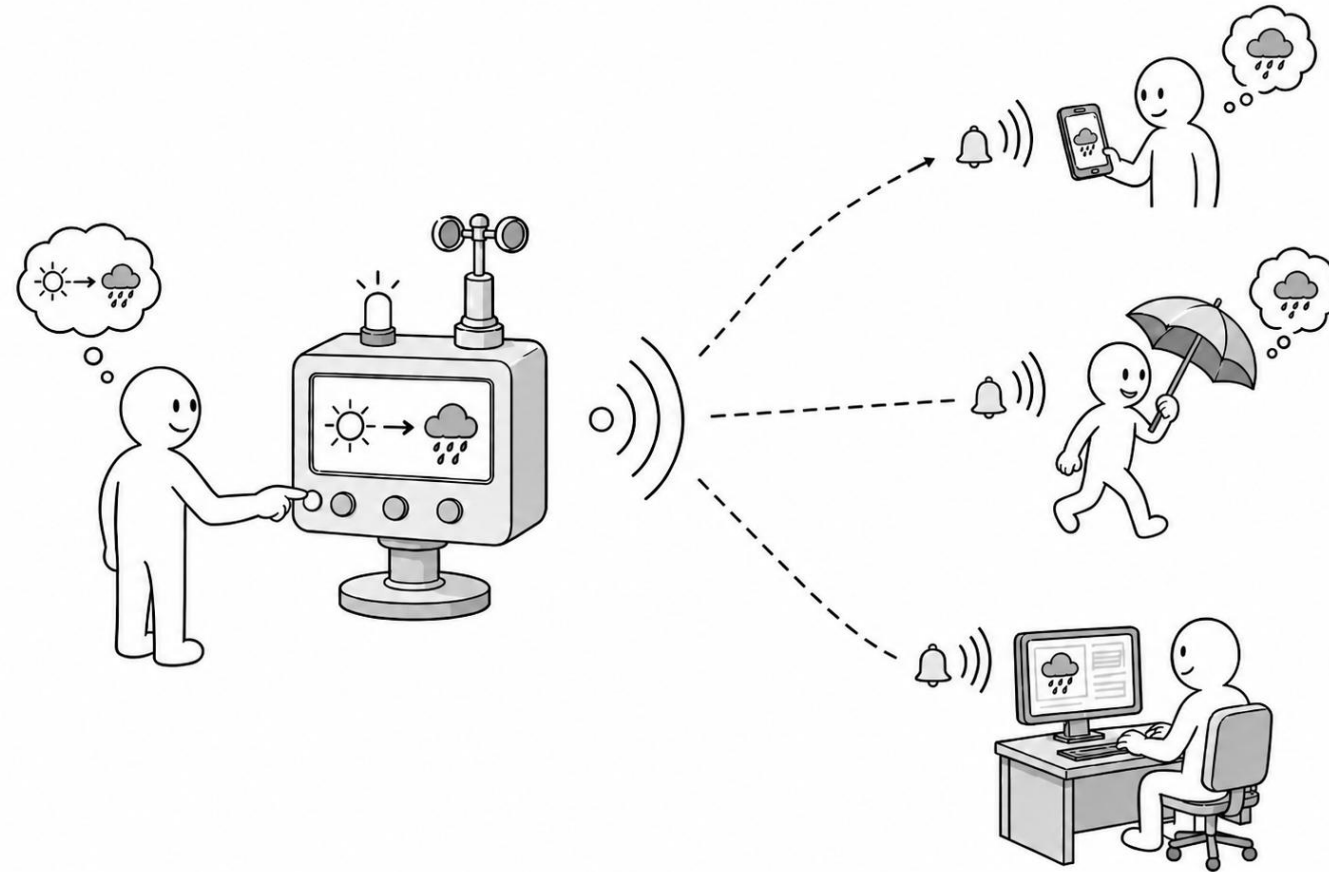
Praxisübung – Kata 06:

> Observer

Observer



Observer



„Ein Objekt benachrichtigt automatisch alle, die sich für seine Änderungen interessieren.“

Pause



Demo – Moderne Praxis

Pause



Abschlussquiz



echolot.app

Q79-EUN

adesso